# Freezing and Flux in a Conserving Combinator Chemistry

Working paper v0.1, Project secret3, August 11, 2026


## Abstract

The conserving reactor of the combinator soup paper is swept along two dials: a temperature, the melting fraction T, the share of cleavage among the aggregation moves, and a pressure, the condensation cap. Sixteen runs of 180,000 ticks each, two seeds on the temperature axis, map the phase behavior with three order parameters per window: reactive flux, copy availability, and species diversity. The results, consistent across seeds: at T = 0 the vessel is the known glass, every molecule unique, zero flux, recruitment starved; at T = 4/5 it is an atom gas, one hundred twenty species, flux collapsed for the opposite reason, nothing left to react; and between them the flux is non-monotonic with its maximum at T = 1/2, where diversity also peaks. The mechanism is decomposed exactly: the volume of attempted duplications falls monotonically with temperature while their success rate rises monotonically from zero to three quarters, carried by copy availability, and the product peaks in the middle. Pressure couples in: tightening the cap at low temperature raises both flux and diversity to their records across the whole sweep, while the gas regime is cap-insensitive. The chemistry of the two moves is most alive, and most diverse, at the balanced melting fraction, starved of substrate below it and of reactants above it.


## 1. Introduction

The soup paper established that the conserving chemistry over S and K lives only when driven: closed, it froze twice into a glass of unique molecules; with reversible condensation it sustained a steady state [1]. That result fixed one point of a dial it did not turn. This paper turns the dial. The question is the one every driven system invites: as the drive varies, where does the vessel freeze, where does it boil away, and is there structure between, a temperature at which the chemistry of the two moves is most alive?

## 2. The dials and the order parameters

Each tick draws an action by integer weights: condensation with weight wCond, cleavage with weight wClv, and a reaction attempt with weight fixed at wCond + wClv, so that reaction attempts are always half of all ticks and points along the axis stay comparable. The temperature is the melting fraction, T = wClv divided by wCond + wClv: the share of taking-apart among the aggregation moves. The baseline chemistry of [1] sits at T equal to one third. The pressure is the condensation cap, the largest molecule two colliders may form. Everything else is as in [1]: genesis from 3,000 S and 3,000 K atoms, recruitment-gated duplication with 24 probes, atom-paying erasure, mass 6,000 verified at every checkpoint of every run, the balances compiled in the shared proof module.

Three order parameters are read per 30,000-tick window, all at the final window after 150,000 ticks of burn-in. Reactive flux is the number of K and S firings in the window. Copy availability is the number of molecules in excess of the number of distinct species, the resource every duplication must find. Diversity is the count of distinct species. Alongside them, the decomposition this paper turns on: the S channel's attempt volume (firings plus blocked attempts) and its success rate.

### 3. The temperature axis

Six temperatures, cap 24, seeds 11 and 77, final window:

| T | flux (seed 11) | flux (seed 77) | diversity (11 / 77) | copy availability (11) | S success rate (11) |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 256 / 262 | 0 | 0.0 percent |
| 1/5 | 1,094 | 1,170 | 322 / 307 | 98 | 3.1 percent |
| 1/3 | 1,629 | 1,558 | 357 / 347 | 198 | 7.3 percent |
| 1/2 | 1,720 | 1,798 | 395 / 385 | 670 | 16.7 percent |
| 2/3 | 805 | 872 | 312 / 317 | 2,615 | 42.8 percent |
| 4/5 | 238 | 251 | 123 / 124 | 4,326 | 66.1 percent |

The anchors first. At T = 0 the vessel reproduces the glass of [1] as a limit of the dial: 256 molecules, every one unique, both atom pools empty, and 14,805 duplication attempts in the final window, every one blocked. At T = 4/5 the vessel is a gas: 4,449 molecules of mean size 1.35, the atom pools holding more than half the mass, diversity collapsed to 123 species, and almost nothing attempts to react, 62 duplication attempts in the window against the glass's fifteen thousand. Between the anchors the flux rises, peaks, and falls, and the peak is at the balanced fraction, T = 1/2, in both seeds. Diversity peaks at the same temperature, 395 and 385 species, the sweep's temperature-axis records. The chemistry is most alive exactly where it is most various.

### 4. The mechanism, decomposed

The non-monotone flux is the product of two monotone curves, and the order parameters separate them cleanly. The attempt volume of the S channel falls with temperature, 14,805 blocked-or-fired at the glass, 9,965 at one fifth, 6,962 at one third, 3,174 at one half, 491 at two thirds, 62 at the gas: hotter vessels hold smaller molecules, and small molecules carry few redexes. The success rate of the S channel rises with temperature, zero at the glass, 3 percent, 7, 17, 43, 66 at the gas, and its carrier is visible in the copy-availability column: melting multiplies copies, and recruitment probability tracks concentration by construction. Flux is attempts times success, a falling curve times a rising one, and the product peaks between. The two dead ends are therefore starvations of opposite kinds, and the paper names them: substrate starvation below the optimum, where redexes abound and copies do not, and reactant starvation above it, where copies abound and redexes do not. The living regime is the negotiation between the two moves' needs: enough aggregation to build the sites where K and S can act, enough melting to supply the material S must spend.

### 5. The pressure axis

Two temperatures at two further caps, seed 11. At T = 1/3, tightening the cap to 16 raises the final-window flux to 2,155 and the diversity to 483 species, both the records of the entire sweep, with copy availability at 422 against the cap-24 value of 198: pressure keeps molecules small, small molecules recur, recurrence feeds recruitment. Loosening the cap to 40 at the same temperature drops the flux to 1,302 and the copy availability to 80, the vessel drifting toward the glass as its mass sinks into large unique boulders. At T = 2/3 the cap scarcely matters, flux 795 to 850 across all three caps: the gas does not feel the ceiling, since almost nothing reaches it. The phase picture is two-dimensional in the expected way: pressure shifts the freezing boundary, and the flux optimum of Section 3 moves toward lower temperature as the cap tightens.

## 6. Discussion

The soup paper's qualitative lesson, that this chemistry must be driven or it dies, is now a curve with a maximum on it. The claim is deliberately local: in this chemistry, with these rules, the flux and the diversity of the driven vessel peak together at the balanced melting fraction, and the mechanism is an exact tradeoff between redex supply and copy supply. Whether the coincidence of the flux peak and the diversity peak is an accident of this rule set or a habit of conserving constructive chemistries is a question the sweep raises and cannot answer; it is the right next question, and the harness that would answer it, other rule sets on the same dials, now exists. The result also sharpens the program's earlier moral in a direction the thermodynamic chapter of the book will want: being driven is not enough, and being driven at the right rate is a real, locatable condition, with too little melting and too much both fatal, for nameably different reasons.

## 7. Limitations

The grid is coarse, six temperatures and three caps, and the optimum is located between one third and two thirds rather than pinned; the refinement is a denser sweep around one half, specified and cheap. Windows are single 30,000-tick reads after burn-in; the seed pair agrees to within a few percent on every reported number, which is the campaign's error bar, and time-averaged windows are the standard upgrade. The reaction share is pinned at half of all ticks by design; other normalizations rescale the axis without, we expect, moving the shape, and checking that expectation is part of the refinement. Mass reads 6,000 at every checkpoint of all sixteen runs.

## 8. Conclusion

A closed vessel of the two moves freezes; an over-melted one evaporates; and between the two starvations the chemistry has a temperature at which it does the most and contains the most, the balanced fraction, where taking-apart matches putting-together. The dial exists, the optimum is real in both seeds, the mechanism decomposes into two monotone curves, and pressure moves the boundary as pressure should. The refinement sweep and the rule-set generalization are the specified continuations.

## Appendix A: Methods

The sweep harness is version 4 of the conserving reactor, Soup2.idr, reproduced in Appendix B: rate weights per channel, network logging switchable and off for the sweep, and a compact per-window reporter of the order parameters with counter deltas. Sixteen runs of 180,000 ticks complete in under a minute and are bit-for-bit reproducible from the stated seeds. The conservation balances are the compiled theorems condMass, kBalanced, and sBalanced of the shared proof module [1].

## Appendix B: The sweep harness

```
||| Project secret3, experiment E6.2/C1 : the conserving reactor,
||| with an operational null (reactions off) and a logged reaction
||| network with food-closure and recycling-core analysis.
||| Conservation balances are compiled in Proofs.idr, part H.
module Soup2

import Data.IOArray
import Data.SortedMap
import Data.List
import Data.String
import Data.Maybe
```

```
import Data.IORef

%default covering

data Tm = Sc | Kc | Ap Tm Tm

render : Tm -> String
render Sc = "S"
render Kc = "K"
render (Ap a b) = render a ++ wrap b
  where wrap : Tm -> String
        wrap Sc = "S"
        wrap Kc = "K"
        wrap t = "(" ++ render t ++ ")"

tsize : Tm -> Nat
tsize Sc = 1
tsize Kc = 1
tsize (Ap a b) = tsize a + tsize b

next64 : Bits64 -> Bits64
next64 s = s * 6364136223846793005 + 1442695040888963407

rnd : Bits64 -> Int -> (Bits64, Int)
rnd s m =
  let s' = next64 s
      v  = (s' `div` 65536) `mod` (cast m)
  in (s', cast v)

data R1 = RK1 | RS1 Tm

redexes : Tm -> List (List Bool, R1)
redexes t0 = go [] t0
  where
    cls : Tm -> Maybe R1
    cls (Ap (Ap Kc _) _) = Just RK1
    cls (Ap (Ap (Ap Sc _) _) c) = Just (RS1 c)
    cls _ = Nothing
    go : List Bool -> Tm -> List (List Bool, R1)
    go p t@(Ap a b) =
      (case cls t of
         Just r => [(reverse p, r)]
         Nothing => [])
      ++ go (False :: p) a ++ go (True :: p) b
    go p _ = []

fireHead : Tm -> (Tm, List Tm)
fireHead (Ap (Ap Kc a) b) = (a, [b, Kc])
fireHead (Ap (Ap (Ap Sc a) b) c) = (Ap (Ap a c) (Ap b c), [Sc])
fireHead t = (t, [])

fireAt : List Bool -> Tm -> (Tm, List Tm)
fireAt [] t = fireHead t
fireAt (False :: p) (Ap a b) = let (a', fr) = fireAt p a in (Ap a' b, fr)
fireAt (True :: p) (Ap a b) = let (b', fr) = fireAt p b in (Ap a b', fr)
```

```
fireAt _ t = (t, [])

-- vessel -------------------------------------------------------------

record Vessel where
  constructor MkV
  arr : IOArray Tm
  nRef : IORef Int

vN : Vessel -> IO Int
vN v = readIORef (nRef v)

vGet : Vessel -> Int -> IO Tm
vGet v i = do
  Just t <- readArray (arr v) i | Nothing => pure Kc
  pure t

vInsert : Vessel -> Tm -> IO ()
vInsert v t = do
  n <- readIORef (nRef v)
  ignore (writeArray (arr v) n t)
  writeIORef (nRef v) (n + 1)

vRemove : Vessel -> Int -> IO ()
vRemove v i = do
  n <- readIORef (nRef v)
  if i == n - 1
    then writeIORef (nRef v) (n - 1)
    else do last <- vGet v (n - 1)
            ignore (writeArray (arr v) i last)
            writeIORef (nRef v) (n - 1)

vRemove2 : Vessel -> Int -> Int -> IO ()
vRemove2 v i j =
  if i > j then do vRemove v i; vRemove v j
           else do vRemove v j; vRemove v i

readAll : Vessel -> IO (List Tm)
readAll v = do
  n <- vN v
  go 0 n
  where go : Int -> Int -> IO (List Tm)
        go i n = if i >= n then pure []
                 else do t <- vGet v i
                         ts <- go (i + 1) n
                         pure (t :: ts)

-- reaction network log -----------------------------------------------

Net : Type
Net = IORef (SortedMap String Nat)

bumpE : Net -> String -> String -> IO ()
bumpE net a b =
  modifyIORef net (\m =>
```

```
    let k = a ++ ">" ++ b
    in case lookup k m of
         Nothing => insert k 1 m
         Just c => insert k (S c) m)

-- reactions -------------------------------------------------------------

record Counters where
  constructor MkC
  cOk, cBlk, clv, clvBlk, kF, sF, sBlk, inert : IORef Nat

bumpR : IORef Nat -> IO ()
bumpR r = modifyIORef r S

probe : Vessel -> Bits64 -> Int -> Nat -> Nat -> String -> IO (Bits64, Maybe Int)
probe v s self Z zsz zr = pure (s, Nothing)
probe v s self (S k) zsz zr = do
  n <- vN v
  let (s1, j) = rnd s n
  if j == self
    then probe v s1 self k zsz zr
    else do u <- vGet v j
            if tsize u == zsz && render u == zr
              then pure (s1, Just j)
              else probe v s1 self k zsz zr

record Params where
  constructor MkP
  capMol : Nat
  probes : Nat
  reactOn : Bool
  logNet : Bool
  wCond : Int
  wClv : Int
  wReact : Int

tick : Params -> Vessel -> Counters -> Net -> Bits64 -> IO Bits64
tick p v c net s = do
  n <- vN v
  let tot = wCond p + wClv p + wReact p
  let (s1, act) = rnd s tot
  if act < wCond p
    then
      if n < 2 then do bumpR (cBlk c); pure s1
      else do
        let (s2, i) = rnd s1 n
        let (s3, j0) = rnd s2 (n - 1)
        let j = if j0 >= i then j0 + 1 else j0
        a <- vGet v i
        b <- vGet v j
        if tsize a + tsize b > capMol p
          then do bumpR (cBlk c); pure s3
          else do
            vRemove2 v i j
            let pr = Ap a b
```

```
          vInsert v pr
          when (logNet p) $ do
            bumpE net (render a) (render pr)
            bumpE net (render b) (render pr)
          bumpR (cOk c)
          pure s3
    else if act < wCond p + wClv p
      then do
        let (s2, i) = rnd s1 n
        t <- vGet v i
        case t of
          Ap a b => do
            vRemove v i
            vInsert v a
            vInsert v b
            when (logNet p) $ do
              bumpE net (render t) (render a)
              bumpE net (render t) (render b)
            bumpR (clv c)
            pure s2
          _ => do bumpR (clvBlk c); pure s2
    else
      if not (reactOn p)
        then do bumpR (inert c); pure s1
        else do
          let (s2, i) = rnd s1 n
          t <- vGet v i
          case redexes t of
            [] => do bumpR (inert c); pure s2
            rs => do
              let (s3, ridx) = rnd s2 (cast (length rs))
              case drop (cast ridx) rs of
                [] => do bumpR (inert c); pure s3
                ((path, RK1) :: _) => do
                  let (t', freed) = fireAt path t
                  vRemove v i
                  vInsert v t'
                  traverse_ (vInsert v) freed
                  when (logNet p) $ do
                    bumpE net (render t) (render t')
                    traverse_ (\u => bumpE net (render t) (render u)) freed
                  bumpR (kF c)
                  pure s3
                ((path, RS1 z) :: _) => do
                  (s4, hit) <- probe v s3 i (probes p) (tsize z) (render z)
                  case hit of
                    Nothing => do bumpR (sBlk c); pure s4
                    Just j => do
                      let (t', freed) = fireAt path t
                      vRemove2 v i j
                      vInsert v t'
                      traverse_ (vInsert v) freed
                      when (logNet p) $ do
                        bumpE net (render t) (render t')
                        bumpE net (render z) (render t')
```

```
                      traverse_ (\u => bumpE net (render t) (render u)) freed
                    bumpR (sF c)
                    pure s4

-- census ----------------------------------------------------------

bump : String -> Nat -> SortedMap String (Nat, Nat) -> SortedMap String (Nat, Nat)
bump k sz m = case lookup k m of
  Nothing => insert k (1, sz) m
  Just (cnt, s0) => insert k (cnt + 1, s0) m

censusMap : List Tm -> SortedMap String (Nat, Nat)
censusMap = foldl (\m, t => bump (render t) (tsize t) m) empty

censusList : SortedMap String (Nat, Nat) -> List (String, Nat, Nat)
censusList m =
  sortBy (\(_, c1, _), (_, c2, _) => compare c2 c1)
    (map (\(r, (cn, sz)) => (r, cn, sz)) (SortedMap.toList m))

shorten : String -> String
shorten str = if length str > 44 then substr 0 44 str ++ ".." else str

cGet : SortedMap String (Nat, Nat) -> String -> Nat
cGet m r = fst (fromMaybe (0, 0) (lookup r m))

printCensus : String -> Vessel -> Counters -> IO ()
printCensus tag v c = do
  pop <- readAll v
  let cm = censusMap pop
  let mass = foldl (\a, t => a + tsize t) 0 pop
  co <- readIORef (cOk c); cb <- readIORef (cBlk c)
  cl <- readIORef (clv c); cbl <- readIORef (clvBlk c)
  kf <- readIORef (kF c); sf <- readIORef (sF c)
  sb <- readIORef (sBlk c); iw <- readIORef (inert c)
  putStrLn $ tag ++ " N=" ++ show (length pop)
    ++ " distinct=" ++ show (length (SortedMap.toList cm))
    ++ " mass=" ++ show mass ++ (if mass == 6000 then "" else " MASS-ERROR")
    ++ " freeS=" ++ show (cGet cm "S") ++ " freeK=" ++ show (cGet cm "K")
  putStrLn $ "  cond=" ++ show co ++ " condBlk=" ++ show cb
    ++ " cleave=" ++ show cl ++ " cleaveBlk=" ++ show cbl
    ++ " kFire=" ++ show kf ++ " sFire=" ++ show sf
    ++ " sBlocked=" ++ show sb ++ " inert=" ++ show iw
  for_ (take 8 (censusList cm)) $ \(r, cn, sz) =>
    putStrLn $ "  " ++ show cn ++ "  sz" ++ show sz ++ "  " ++ shorten r

printFinal : String -> Vessel -> IO ()
printFinal tag v = do
  pop <- readAll v
  let cm = censusMap pop
  let rows = filter (\(_, cn, _) => cn >= 2) (censusList cm)
  putStrLn $ tag ++ " final species table (count >= 2), "
    ++ show (length rows) ++ " species:"
  for_ (take 24 rows) $ \(r, cn, sz) =>
    putStrLn $ "  " ++ show cn ++ "  sz" ++ show sz ++ "  " ++ shorten r
```

```
-- network analysis ------------------------------------------------

splitEdge : String -> (String, String)
splitEdge s =
  let (a, b) = break (== '>') s
  in (a, pack (drop 1 (unpack b)))

addAdj : String -> String -> SortedMap String (List String) ->
         SortedMap String (List String)
addAdj a b m = case lookup a m of
  Nothing => insert a [b] m
  Just xs => if elem b xs then m else insert a (b :: xs) m

bfs : SortedMap String (List String) -> List String ->
      SortedMap String Bool -> SortedMap String Bool
bfs adj [] vis = vis
bfs adj (x :: xs) vis =
  case lookup x vis of
    Just _ => bfs adj xs vis
    Nothing =>
      let nbrs = fromMaybe [] (lookup x adj)
      in bfs adj (nbrs ++ xs) (insert x True vis)

analyzeNet : String -> Vessel -> Net -> IO ()
analyzeNet tag v net = do
  m <- readIORef net
  let es = map (\(k, c) => let (a, b) = splitEdge k in (a, b, c))
               (SortedMap.toList m)
  let fwd = foldl (\acc, (a, b, _) => addAdj a b acc)
                  (the (SortedMap String (List String)) empty) es
  let rev = foldl (\acc, (a, b, _) => addAdj b a acc)
                  (the (SortedMap String (List String)) empty) es
  let fc = bfs fwd ["S", "K"] (the (SortedMap String Bool) empty)
  let rc = bfs rev ["S", "K"] (the (SortedMap String Bool) empty)
  pop <- readAll v
  let cm = censusMap pop
  let species = censusList cm
  let unreach = filter (\(r, _, _) => isNothing (lookup r fc)) species
  let core = filter (\(r, _, _) =>
                isJust (lookup r fc) && isJust (lookup r rc)) species
  putStrLn $ tag ++ " network: edges=" ++ show (length es)
    ++ " speciesNow=" ++ show (length species)
    ++ " foodReachable=" ++ show (length species `minus` length unreach)
    ++ " unreachable=" ++ show (length unreach)
    ++ " recyclingCore=" ++ show (length core)
  putStrLn $ tag ++ " recycling core, top by abundance:"
  for_ (take 14 core) $ \(r, cn, sz) =>
    putStrLn $ "  " ++ show cn ++ "  sz" ++ show sz ++ "  " ++ shorten r
  let tops = take 10 (sortBy (\(_, _, c1), (_, _, c2) => compare c2 c1) es)
  putStrLn $ tag ++ " top edges by flux:"
  for_ tops $ \(a, b, c) =>
    putStrLn $ "  " ++ show c ++ "  " ++ shorten a ++ " > " ++ shorten b

runSweep : String -> Params -> Bits64 -> IO ()
runSweep label p s0 = do
```

```
  arr0 <- newArray 6100
  nR <- newIORef 0
  let v = MkV arr0 nR
  for_ (the (List Int) [0..2999]) $ \_ => vInsert v Sc
  for_ (the (List Int) [0..2999]) $ \_ => vInsert v Kc
  r1 <- newIORef 0; r2 <- newIORef 0; r3 <- newIORef 0; r4 <- newIORef 0
  r5 <- newIORef 0; r6 <- newIORef 0; r7 <- newIORef 0; r8 <- newIORef 0
  let c = MkC r1 r2 r3 r4 r5 r6 r7 r8
  net <- newIORef empty
  go v c net s0 0 6 0 0 0
  where
    loop : Vessel -> Counters -> Net -> Bits64 -> Nat -> IO Bits64
    loop v c net s Z = pure s
    loop v c net s (S k) = do s' <- tick p v c net s; loop v c net s' k
    go : Vessel -> Counters -> Net -> Bits64 -> Nat -> Nat ->
         Nat -> Nat -> Nat -> IO ()
    go v c net s done Z pk ps pb = pure ()
    go v c net s done (S more) pk ps pb = do
      s' <- loop v c net s 30000
      let done' = done + 30000
      pop <- readAll v
      let cm = censusMap pop
      let mass = foldl (\a, t => a + tsize t) 0 pop
      kf <- readIORef (kF c); sf <- readIORef (sF c)
      sb <- readIORef (sBlk c)
      putStrLn $ label ++ " t=" ++ show done'
        ++ " N=" ++ show (length pop)
        ++ " dist=" ++ show (length (SortedMap.toList cm))
        ++ " dup=" ++ show (length pop `minus` length (SortedMap.toList cm))
        ++ " freeS=" ++ show (cGet cm "S") ++ " freeK=" ++ show (cGet cm "K")
        ++ " dkF=" ++ show (kf `minus` pk) ++ " dsF=" ++ show (sf `minus` ps)
        ++ " dsBlk=" ++ show (sb `minus` pb)
        ++ (if mass == 6000 then "" else " MASS-ERROR")
      go v c net s' done' more kf sf sb

main : IO ()
main = do
  let pts = the (List (Int, Int, String))
        [(4, 0, "T=0/4"), (4, 1, "T=1/5"), (4, 2, "T=1/3"),
         (4, 4, "T=1/2"), (2, 4, "T=2/3"), (1, 4, "T=4/5")]
  for_ (the (List Bits64) [11, 77]) $ \sd =>
    for_ pts $ \(wc, wv, lb) =>
      runSweep ("SW cap24 " ++ lb ++ " seed" ++ show sd)
        (MkP 24 24 True False wc wv (wc + wv)) sd
  for_ (the (List Nat) [16, 40]) $ \cp =>
    for_ (the (List (Int, Int, String))
          [(4, 2, "T=1/3"), (2, 4, "T=2/3")]) $ \(wc, wv, lb) =>
      runSweep ("SW cap" ++ show cp ++ " " ++ lb ++ " seed11")
        (MkP cp 24 True False wc wv (wc + wv)) 11
```

## References


1. Project secret3, ‘The Combinator Soup: Monoculture Collapse onto Self-Reproducing Absorbers, and the Conserving Reactor That Extinguishes Them’, working paper v0.3, August 11, 2026.

2. Project secret3, ‘A Research Agenda for the Two Moves: Companion Papers to the Book’, research agenda v0.4, August 11, 2026.
3. W. Fontana and L. W. Buss, “The arrival of the fittest’: toward a theory of biological organization’, Bulletin of Mathematical Biology 56 (1994), 1-64.
4. G. Kruszewski and T. Mikolov, ‘Combinatory chemistry: towards a simple model of emergent evolution’, Proceedings of ALIFE 2020, MIT Press, 2020.
5. E. Brady, ‘Idris 2: Quantitative Type Theory in Practice’, *Proceedings of ECOOP 2021*, LIPIcs 194, 2021.